Libertya REST API

Manual para desarrolladores

Indiced

Indice
Introducción
Fuentes
Entorno de desarrollo
Generalidades para el desarrollo
- Definición de la API
  - Definición principal
  - Definición de end-points
  - Definición de schemas
  - Script generdor de schemas
- Generación de clases mediante Swagger CodeGen
- Implementación de clases

Ejemplo
- Ampliación de paths principales
- Ampliación de endpoints
- Ampliación del schema (autogenerado)
- Ampliación del schema (manual)
- Generación de clases
- Implementación de repository
- Implementación de service
- Implementación de controller

Casos especiales
Testings de integración
Build del proyecto
- Release para LY CORE 22.0ar
- Consideraciones para LY CORE 26.0 o superiores
  - Soporte Java 11
  - Soporte Postgres 16
- Creacion de docker image

Uso

Introducción

El presente documento detalla los pasos para desarrollar y ampliar Libertya REST API, la cual fue desarrollada bajo Spring Boot en conjunto con Swagger OpenAPI.

Fuentes

Los fuentes de LY REST API se alojan en el siguiente repositorio de github:

- https://github.com/Disytel-Consulting-SA/lyrestapi

Entorno de desarrollo

Si bien pueden utilizarse otros entornos, el desarrollo de LY REST API se realizó utilizando **Intellij IDEA**.

El proyecto se apoya en las librerías **OXP.jar** y **OXPXLib.jar** según la definición de **build.grade**, con lo cual es necesario contar con la variable de entorno **OXP_HOME** definida previo inicio del IDE a fin de que encuentre dichas librerías para la compilación del proyecto.

```
def OXPLIBS = System.getenv("OXP_HOME")
implementation files("$OXPLIBS/lib/OXP.jar")
implementation files("$OXPLIBS/lib/OXPXLib.jar")
```

El proyecto requiere además **Java 8**.

Generalidades para el desarrollo

Definición de la API

Definición principal

El archivo **src/main/resources/ly-rest-api.yaml** es el punto de inicio para la ampliacion de operaciones. En dicho archivo se definen los paths a los distintos archivos que contienen los end-points:

```yaml
paths:
 /v1.0/products:
   $ref: ./paths/products.yaml
 /v1.0/products/{id}:
   $ref: ./paths/products_id.yaml
 /v1.0/bpartners:
   $ref: ./paths/bpartners.yaml
 /v1.0/bpartners/{id}:
   $ref: ./paths/bpartners_id.yaml
 /v1.0/invoices:
   $ref: ./paths/invoices.yaml
 /v1.0/invoices/{id}:
   $ref: ./paths/invoices_id.yaml
 /v1.0/invoices/{id}/process:
   $ref: ./paths/invoices_id_process.yaml
```

En general, para un tipo de entidad tendremos 2 entradas, por ejemplo:

- **/v1.0/products:**
  - products.yaml
- **/v1.0/products/{id}:**
  - producs_id.yaml

La primera entrada se encuentra relacionada con operaciones que no requieren indicar el identificador, como por ejemplo creación de una nueva entrada o bien listados. La segunda entrada sí requiere un identificador y se utiliza para operaciones como modificación, eliminación o recuperación de una entidad en particular.

NOTA: Existirá una tercera entrada en ciertos casos, para procesamiento de entidades de tipo documento (pedidos, facturas, remitos, etc.), por ejemplo:

- **/v1.0/invoices/{id}/process:**
  - $ref: ./paths/invoices_id_process.yaml

Definición de end-points

Los archivos **src/main/resources/paths/*.yaml** contienen los endpoints para cada una de las entidades sobre las cuales operar, por ejemplo **bpartners.yaml** / **bpartners_id.yaml**:

```
get:
 tags:
   - "bpartner"
 summary: Recupera una entidad comercial en particular
 parameters:
   - name: id
     in: path
     description: ID de la entidad comercial
     required: true
     schema:
       type: integer
 description: Recupera la informacion de una entidad comercial en particular
 operationId: retrieveBPartner
 responses:
   "200":
     description: OK
     content:
       application/json:
         schema:
           $ref: '../model/bpartner.yaml#/components/schemas/BPartner'
 ...
```

Definición de schemas

Los archivos **src/main/resources/model/*.yaml** contienen los schemas referenciados en los end-points y <u>son autogenerados</u> mediante un script (se detalla luego) basándose en los metadatos de **AD_Table**, **AD_Column**.

```
components:
 schemas:
   BPartner:
     type: object
     properties:
       ad_client_id:
         type: integer
       ad_componentobjectuid:
         type: string
        ...
```

```
        value:
          type: string
```

NOTA: Dentro de **src/main/resources/model** existen también los archivos ***_doc.yaml** (por ejemplo **invoice_doc.yaml** o **order_doc.yaml**), los cuales no son autogenerados. Estos archivos representan una estructura más amplia que una entidad, o sea un documento que abarca varias entidades. Por ejemplo una factura abarcará sus líneas y sus impuestos:

```
components:
 schemas:
   InvoiceDocument:
     type: object
     properties:
       header:
         $ref: '../model/invoice.yaml#/components/schemas/Invoice'
       lines:
         type: array
         items:
           $ref: '../model/invoiceline.yaml#/components/schemas/InvoiceLine'
       taxes:
         type: array
         items:
           $ref: '../model/invoicetax.yaml#/components/schemas/InvoiceTax'
```

Notar que la definición de **invoice_doc.yaml** se apoya en los archivos yaml de modelo autogenerados.

Adicionalmente, en **src/main/resources/patrh** existen los archivos ***_doc_process.yaml** para el procesado de estos documentos (completar, anular, revertir, cerrar, etc.).

```
put:
  tags:
    - "invoice"
  summary: Procesa una factura
  operationId: processInvoice
  parameters:
    - in: path
      name: id
      description: ID de la factura a procesar
      required: true
      schema:
        type: integer
    - in: query
      name: action
      required: true
      description: Accion a aplicar (completar, revertir, etc.)
      schema:
        type: string
...
```

Script generdor de schemas

El script **utils/genSchema.sh** es el encargado de (re)generar los archivos **yaml** descriptores del modelo ubicados en **src/main/resources/model**.

Es posible ampliar el apartado **#Scripts de generacion** con la/s nueva/s tabla/s que se requieran, por ejemplo:

```
generateSchema Currency   C_Currency   currency.yaml
```

En donde **Currency** es el nombre del schema a generar, **C_Currency** es el nombre de la tabla sobre la cual obtener la estructura a generar y **currency.yaml** es el nombre del archivo destino a crear.

Adicionalmente, es posible limitar las propiedades a incluir en el schema, indicando una lista de columnas relevantes a considerar (además de las obligatorias según **ismandatory** en **ad_column** para dichas columnas), e ignorar el resto. Para ésto debe incorporarse un cuarto argumento en la invocación con las columnas en cuestión:

```
generateSchema Currency   C_Currency   currency.yaml  "('description', 'iseuro',
'wsfecode')"
```

De no especificar la lista de columnas, **todas** las columnas serán consideradas, excepto las de tipo binarias (ver **genSchema.sql** para más detalles).

IMPORTANTE: Debe especificarse además los datos de la conexion a la base de datos en el apartado **# DB Connection**.

Generación de clases mediante Swagger CodeGen

El script **utils/genClasses.sh** es el encargado de generar las interfaces del package **org.libertya.api.stub.iface** (por ejemplo *org.libertya.api.stub.iface.ProductApi*) y las clases de modelo del package **org.libertya.api.stub.model** (por ejemplo *org.libertya.api.stub.model.Product*) a partir de las definiciones previamente especificadas en los archivos yaml (los definidos manualmente más los autogenerados con el script **genSchema.sh**).

Para facilitar el workflow de desarrollo, el script **genClasses.sh** directamente ejecuta **genSchema.sh** como primera actividad.

Implementación de clases

Las clases a implementar para operaciones de tipo CRUD o procesamiento de documentos son básicamente 2 (o 3 si la entidad es un documento):

- El repository, por ejemplo **org.libertya.api.repository.InvoiceRepository**
- El repository, por ejemplo **org.libertya.api.repository.InvoiceService** (si la entidad es un documento o si se requiere lógica adicional)
- El controller, por ejemplo **org.libertya.api.controller.InvoiceController**

Se debe implementar la interfaz generada a fin de implementar los Controllers correspondientes, los cuales recibirán y retornaran los tipos definidos en el modelo. Por ejemplo la clase **BPartnerController** implementa **BpartnerApi**, y en los distintos métodos se envía/reciben clases de tipo **org.libertya.api.stub.model.BPartner** (clase autogenerada).

Ejemplo

A modo de ejemplo se detalla el paso a paso para dar soporte a la gestión de remitos (InOuts), abarcando:

- Creación
- Eliminación
- Modificación
- Recuperación
- Procesado

Ampliación de paths principales

El primer paso es incluir a los nuevos paths relacionados con la gestión de remitos en el archivo **ly-rest-api.yaml**:

```
/v1.0/inouts:
 $ref: ./paths/inouts.yaml
/v1.0/inouts/{id}:
 $ref: ./paths/inouts_id.yaml
/v1.0/inouts/{id}/process:
 $ref: ./paths/inouts_id_process.yaml
```

Ampliación de endpoints

Se deben incorporar los 2 o 3 archivos (si la entidad tiene lógica de documentos como es el caso de remitos) de definicion de operaciones:

- **src/main/resources/paths/inouts.yaml**
  - Para listar o crear remitos (no requieren un id como parte del path)
- **src/main/resources/paths/inouts_id.yaml**
  - Para recuperar un remito o bien para modificar o eliminar un remito (requiere su id como parte del path)
- **src/main/resources/paths/inouts_id_process.yaml**
  - Para completar, anular, cerrar, etc. un remito (requiere su id como parte del path)

Por ejemplo, para **inouts.yaml** el end-point **GET** es el siguiente (ver contenidos completos en el proyecto):

```
get:
 tags:
   - "inout"
 summary: Retrieve inouts
 description: Retorna una lista de remitos
```

```
 operationId: getAllInOuts
...
```

Es importante respetar los tags en todos los casos dentro de las operaciones de los 3 archivos, a fin de que cada entidad sea luego generada bajo su interfaz exclusiva correspondiente.

Las definiciones de estos archivos son similares, con lo cual pueden ser basados en otros ya creados, por ejemplo **orders.yaml**, **orders_id.yaml**, **orders_id_process.yaml**. Cabe destacar que ante el desarrollo inicial para una entidad dada, estos archivos referenciarán a schemas todavía no existentes dentro del directorio **src/main/resources/model**. La generación de estos archivos de detalla en el apartado a continuación.

Ampliación del schema (autogenerado)

Se debe incluir el modelo **InOut** en la nómina de esquemas basado en la información de **M_InOut**. De manera similar para **M_InOutLine**. En el archivo **genSchema.sh**, en el apartado **#Script de generacion** incorporar:

```
generateSchema InOut          M_InOut           inout.yaml
generateSchema InOutLine      M_InOutLine       inoutline.yaml
```

Si se ejecuta **genSchema.sh** veremos que se crean los archivos **src/main/resources/model/inout.yaml** y **src/main/resources/model/inoutline.yaml** correspondientes:

```
components:
 schemas:
   InOut:
     type: object
     properties:
       ad_client_id:
         type: integer
       ad_org_id:
         type: integer
```

Ampliación del schema (manual)

Para entidades de tipo documento, se debe crear manualmente el descriptor de schema **src/main/resources/model/inout_doc.yaml** el cual abarca la cabecera y las lineas:

```
components:
 schemas:
   InOutDocument:
     type: object
     properties:
       header:
         $ref: '../model/inout.yaml#/components/schemas/InOut'
       lines:
         type: array
         items:
           $ref: '../model/inoutline.yaml#/components/schemas/InOutLine'
```

Generación de clases

Ejecutar **genClasses.sh**, el cual generará tanto las clases de modelo como la interfaz de la API para la entidad correspondiente:

- Modelo
  - **org.libertya.api.stub.model.InOut**
  - **org.libertya.api.stub.model.InOutLine**
- API
  - **org.libertya.api.stub.iface.InOutApi**

Implementación de repository

Se debe implementar el repository correspondiente para los remitos, llamado **InOutRepository**. Todos los repositories deben extender de la clase **AbstractRepository**, la cual tiene las facilidades en común para todas las subclases.

Toda la lógica para la gestión de entidades y mapeo a PO radica en **AbstractRepository**. Por consiguiente, la clase **InOutRepository** es muy sencilla, solo es necesario indicar que es un **@Repository** e incorporar un constructor indicando el nombre de la tabla y el método para la instanciación de objetos de dicho tipo.

```
@Repository
public class InOutRepository extends AbstractRepository {

   public InOutRepository() {
       tableName = X_M_InOut.Table_Name;
       iface = InOut::new;
   }
}
```

Los métodos retrieve (en todas sus variedades), delete, update, delete, process son gestionados por la superclase.

De manera similar, se debe implementar el repository InOutLineRepository:

```
@Repository
public class InOutLineRepository extends AbstractRepository {

   public InOutLineRepository() {
       tableName = X_M_InOutLine.Table_Name;
       iface = InOutLine::new;
   }
}
```

Implementación de service

Las entidades que contemplan el scope de documento deberán además implementar la clase de servicio correspondiente para la gestión de procesado de documento, así como operaciones de recuperación del documento completo (abarcando por ejemplo sus líneas). En este caso, se deberá implementar **InOutService**, la cual deberá extender de **AbstractService**.

Los métodos a implementar son 3:

- **getRepository()** el cual debe retornar el repositorio que gestiona la cabecera del documento (en este caso **InOutRepository**).
- **performCreate()** conteniendo la lógica de creación de la cabecera, líneas, etc.
- **performRetrieve()** conteniendo la lógica de recuperación de la cabecera, líneas, etc.

Por ejemplo, para la creacion de un documento de tipo remito:

```
@Override
protected String performCreate(UserInfo info, Object document, String trxName) throws
Exception {
   InOutDocument InOutDocument = (InOutDocument)document;

   // Cabecera
   Integer id = Integer.parseInt(inoutRepository.insert(info, InOutDocument.getHeader(),
trxName));

   // Lineas
   for (InOutLine InOutLine : getList(InOutDocument.getLines())) {
       InOutLine.setMInoutId(id);
       inoutLineLineRepository.insert(info, InOutLine, trxName);
   }

   return Integer.toString(id);
}
```

La superclase **AbstractService** se encargará de crear y commitear la transacción, y en caso de error realizar el rollback correspondiente.

Implementación de controller

Se debe implementar el controller correspondiente para los remitos, llamado **InOutController**, el cual debe implementar **InoutApi** (autogenerada). **InOutController** interactua con **InOutRepository / InOutService** y debe extender de **AbstractController**, la cual tiene facilidades en común para todos los controllers.

El controller **InOutController** se apoyará tanto en **InOutRepository** como en **InOutService** a fin de responder a las operaciones add, delete, update, etc..

```
@Controller
@RequiredArgsConstructor
public class InOutController extends AbstractController implements InoutApi {

   private final HttpServletRequest request;

   private final InOutRepository repository;

   private final InOutService service;

   @Override
   public ResponseEntity<String> addInOut(InOutDocument body) {
       return insertAction(request, (info) -> service.create(info, body));
   }
```

```
   @Override
   public ResponseEntity<String> deleteInOut(Integer id) {
       return deleteAction(request, (info) -> repository.delete(info, id));
   }
...
```

Casos especiales

Ciertos circuitos requieren como entrada información completamente distinta a la que luego es almacenada en base de datos. El caso más común es el de OP/RC en donde se requiere la nómina de facturas a pagar y la nómina de pagos, sin información adicional, por ejemplo una estructura del tipo:

```
{
      "c_bpartner_id": 1012142,
      "earlypayment": false,
      "invoices":
      [
            {
                  "c_invoice_id": "1022046",
                  "amount": "1"
            }
      ],
      "payments":
      [
            {
                  "c_pospaymentmedium_id": "1010269",
                  "amount": "1",
                  "c_payment_id": "1012189"
            }
      ]
}
```

Si bien para las operaciones de recuperacion, eliminación, etc. las definiciones de modelo y clases autogeneradas son de utilidad, específicamente **para la generación de las OP/RC se requiere una estructura específica**. La misma se define bajo **model/allocation_new.yaml** y es referenciada para la operacion POST del endpoint de allocations. De esta manera es posible que “convivan” tanto el modelo autogenerado como incorporaciones adicionales específicas que sean requeridas.

Testings de integración

A fin de contar con una clase que automáticamente valide la correcta ejecución del código asociado al circuito implementado, es posible crear una clase que cuente con una serie de tests asociados. La clase **InOutIntegrationTests** extenderá de **CommonIntegrationTest**, y en ella deberán incorporase los distintos casos de pruebas, tales como creación de un remito, eliminación, modificación, etc. Por ejemplo un test para validar la correcta creación de un remito:

```
@Test
```

```
@Order(1)
void createInOutSouldReturnOK() throws Exception {
   ResponseEntity<String> response =
           restTemplate.exchange(getBaseURL("v1.0/inouts"),
                   HttpMethod.POST,
                   new HttpEntity<>(getInOutContent(), getAuthHeaders()),
                   String.class);
   System.out.println(response.getBody());
   assertThat(response.getStatusCode().toString()).contains("200");
   documentID = Integer.parseInt(response.getBody());
   assertThat(documentID>0);
}
```

La superclase **CommonIntegrationTest** se encarga de la lógica en común a todos estos tipos de test como es la gestion del puerto para testing, la obtención del token, etc.

Luego pueden ejecutarse los tests creados a fin de validar su correcta ejecución.

InOutIntegrationTests

all > org.libertya.api > InOutIntegrationTests

10	0	0	2.007s
tests	failures	ignored	duration

100% successful

Tests | Standard output | Standard error

Test	Duration	Result
createInOutSouldReturnOK()	1.449s	passed
deleteInOutWithCompanyMismatchShouldReturnKO()	0.026s	passed
deleteInOutWithWrongTokenShouldReturnKO()	0.026s	passed
deleteInexistentInOutShouldReturnKO()	0.027s	passed
deleteProcessedInOutShouldReturnKO()	0.120s	passed
modifyInOutShouldReturnOK()	0.087s	passed
modifyInexistentInOutShouldReturnKO()	0.024s	passed
newTokenWithCorrectCredentialsShouldReturnOK()	0.035s	passed
newTokenWithIncorrectCredentialsShouldReturnKO()	0.013s	passed
retrieveCreatedInOutShouldReturnOK()	0.200s	passed

Build del proyecto

Dentro de las tasks de gradle, ejecutar la task **build**. También se puede realizar desde terminal mediante el comando ***./gradlew build***.

Esto generará el archivo **build/libs/lyrestapi-0.0.1-SNAPSHOT.jar** (o la versión que corresponda según la definición de version en **build.gradle**).

Notar que dentro del jar, en **/BOOT-INF/lib/** se encuentran las librerías **OXP.jar** y **OXPXLib.jar** utilizadas en el build del proyecto. Por consiguiente, en caso de actualizar la versión de Libertya CORE (o sus componentes), será necesario reemplazar estos archivos con las nuevas versiones.

NOTA: Tener en cuenta que el build disparará el test-set contenido en **src/test/java/oprg.libertya.api** lo cual puede demorar un tiempo considerable en ejecutar todas las pruebas.

Release para LY CORE 22.0ar

LY REST API se apoya en ciertos cambios de CORE posteriores al release 22.0, con lo cual fue necesario realizar los siguientes pasos:

La generacion de **lyrestapi-1.0.0_for_LY22.0ar.jar** se realizó de la siguiente manera:

Tomar ServidorOXP_V22.0.zip, descomprimir.

Aplicar el patch **org.libertya.core.22.0.patches.a101ec1.99b34e5.jar**, conteniendo los siguientes commit IDs:

- **a101ec1** (DocumentEngine)
- **99b34e5** (DB)

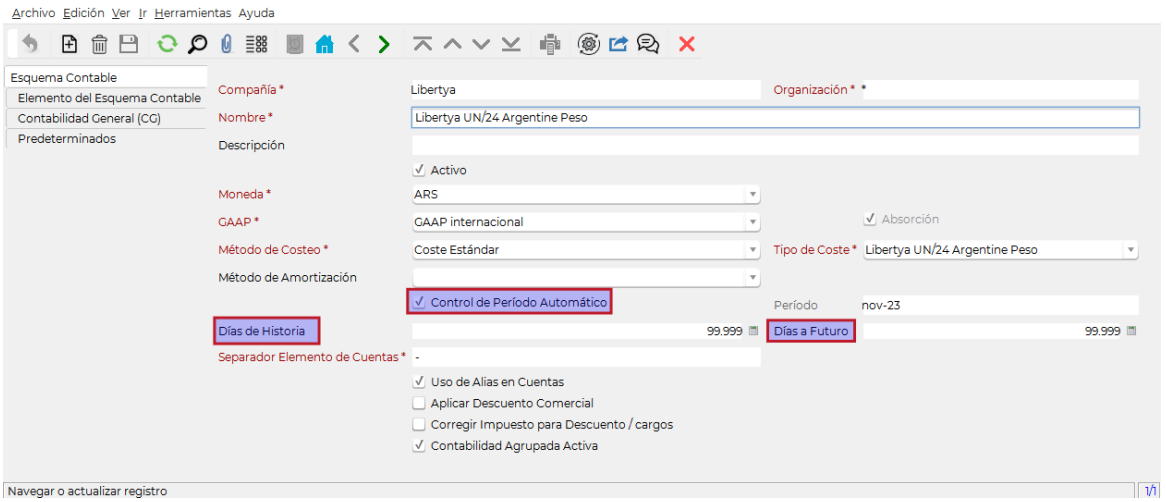
Reconfigurar la instancia ServidorOXP con dicho patch, lo cual generará el **OXP.jar** y **OXPXLib.jar** que el proyecto LYRESTAPI referencia.

Desde Intellij Idea, refrescar y buildear la app, lo cual genera el jar **lyrestapi-1.0.0.jar** generado en **build/libs**

Renombrado el jar a: **lyrestapi-1.0.0_for_LY22.0ar.jar**

NOTA: En caso de no aplicar los patches o de utilizar los binarios de una versión antigua o distinta a LY 22.0, la compilación puede fallar. Esto puede manifestarse por ejemplo con un error al querer compilar **ErrorController**, en donde se presenta el mensaje **Cannot resolve symbol 'ERROR_STATUS_CODE'**.

NOTA: Si el build falla es probablEse que algún test no supere la validación. Puede deberse a que en la base de datos el período no esté abierto. Si la misma tiene control automático de período, entonces poner -9999 y 9999 en fecha hacia atras ya hacia adelante.



Consideraciones para LY CORE 26.0 o superiores

Soporte Java 11

Para versiones que se apoyen en una versión de Libertya que brinda soporte a **Java 11**, deberá realizarse la adecuación detallada a continuación.

La revisión **a0e93ef** de LY CORE incluye un conjunto de modificaciones para dar soporte a Java 11, entre ellas la inclusión de nuevas librerías de Jacorb para reemplazar módulos de Java EE y CORBA:

```
<!-- En JDK 11, los modulos Java EE y CORBA fueron removidos. Se usa Jacorb en su reemplazo. jacorg.org -->
<unjar src="lib/antlr-2.7.2.jar" dest="construirOXPLIB" />
<unjar src="lib/idl.jar" dest="construirOXPLIB" />
<unjar src="lib/jacorb-3.9.jar" dest="construirOXPLIB" />
<unjar src="lib/jacorb-omgapi-3.9.jar" dest="construirOXPLIB" />
<unjar src="lib/jacorb-services-3.9.jar" dest="construirOXPLIB" />
<unjar src="lib/picocontainer-1.2.jar" dest="construirOXPLIB" />
<unjar src="lib/slf4j-api-1.7.14.jar" dest="construirOXPLIB" />
<unjar src="lib/slf4j-jdk14-1.7.14.jar" dest="construirOXPLIB" />
<unjar src="lib/wrapper-3.1.0.jar" dest="construirOXPLIB" />
```

Esto genera un conflicto con las librerías usadas en el proyecto LYRESTAPI, generando el siguiente mensaje de error:

```
java.lang.IllegalArgumentException: LoggerFactory is not a Logback LoggerContext but Logback is on the classpath. Either remove Logback or the competing implementation (class org.slf4j.impl.JDK14LoggerFactory loaded from file:/ServidorOXP/lib/OXPXLib.jar). If you are using WebLogic you will need to add 'org.slf4j' to prefer-application-packages in WEB-INF/weblogic.xml: org.slf4j.impl.JDK14LoggerFactory
```

La manera más sencilla de resolver este problema es simplemente modificar el archivo **OXPXLib.jar** que se está referenciando y eliminar el directorio **org/slf4j/impl**.

Tener en cuenta que la eliminación puede traer problemas con el META-INF/INDEX.LIST. Esto puede solucionarse eliminando también el index. Por ejemplo para incluir en un script:

```
zip -d OXPXLib.jar "org/slf4j/impl/*"
zip -d OXPXLib.jar "META-INF/INDEX.LIST"
```

Una vez realizado esto, en Intellij Idea ir a **File → Reload All From Disk** para recargar los cambios en el entorno. De esta manera el error queda resuelto dado que ya no existirá el mencionado conflicto.

Soporte Postgres 16

Postgres 16 utiliza scram-sha-256 como método de encriptación, el cual no es soportado por versiones viejas de JDBC. Esto genera la imposibilidad de conectar contra una base de datos que **no** utilice trust como en el archivo **hba.conf**, bajo el mensaje de error: **psql: authentication method 10 not supported**.

La solución implica incorporar una versión reciente de JDBC, como por ejemplo la 42.7.7 en los binarios de LY CORE que están siendo referenciados por la API. El driver **postgresql.jar** descargado desde el sitio

https://jdbc.postgresql.org/download/ debe ser ubicado en **/ServidorOXP/lib**, pisando la versión vieja y luego reconfigurar.

Si bien esta librería ya fue incorporada a los fuentes de LY CORE, versiones anteriores contendrán una versión desactualizada del JDBC y por lo tanto será necesario realizar los pasos aquí mencionados.

Creacion de docker image

El proyecto cuenta con el script **DockerCreateImage.sh** que genera una imagen de la aplicación apoyada en **OpenJDK 8**.

```
usuario@pc-usuario:~/workspace/org.libertya.api$ ./DockerCreateImage.sh
BUILD SUCCESSFUL in 2s
7 actionable tasks: 7 up-to-date
[+] Building 2.0s (7/7) FINISHED
 => [internal] load .dockerignore
0.1s
 => => transferring context: 2B
0.0s
 => [internal] load build definition from Dockerfile
0.1s
 => => transferring dockerfile: 172B
0.0s
 => [internal] load metadata for docker.io/library/openjdk:8-jdk-alpine
0.0s
 => [internal] load build context
0.7s
 => => transferring context: 67.80MB
0.6s
 => [1/2] FROM docker.io/library/openjdk:8-jdk-alpine
0.1s
 => [2/2] COPY build/libs/lyrestapi-1.0.0.jar lyrestapi-1.0.0.jar
0.6s
 => exporting to image
0.5s
 => => exporting layers
0.5s
 => => writing image
sha256:b155879ce2ab4dc1c89b6a287753435ba69aae268bc7c818a0e5bbce4e9192c3
0.0s
 => => naming to docker.io/library/lyrestapi
0.0s

usuario@pc-usuario:~/workspace/org.libertya.api$ docker images
REPOSITORY                                      TAG              IMAGE ID       CREATED
SIZE
lyrestapi                                       latest           b155879ce2ab   About a
minute ago   173MB
```

Uso

Ver el documento Libertya REST API - Manual de uso.